

```

1  """
2  Starter code for the problem "Cart-pole swing-up".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  import time
8
9  from animations import animate_cartpole
10
11 import jax
12 import jax.numpy as jnp
13
14 import matplotlib.pyplot as plt
15
16 import numpy as np
17
18 from scipy.integrate import odeint
19
20
21 def linearize(f, s, u):
22     """Linearize the function `f(s, u)` around `(s, u)`.
23
24     Arguments
25     -----
26     f : callable
27         A nonlinear function with call signature `f(s, u)`.
28     s : numpy.ndarray
29         The state (1-D).
30     u : numpy.ndarray
31         The control input (1-D).
32
33     Returns
34     -----
35     A : numpy.ndarray
36         The Jacobian of `f` at `(s, u)`, with respect to `s`.
37     B : numpy.ndarray
38         The Jacobian of `f` at `(s, u)`, with respect to `u`.
39     """
40     # WRITE YOUR CODE BELOW #####
41     # INSTRUCTIONS: Use JAX to compute `A` and `B` in one line.
42     A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
43     #####
44     return A, B
45
46
47 def ilqr(f, s0, s_goal, N, Q, R, QN, eps=1e-3, max_iters=1000):
48     """Compute the iLQR set-point tracking solution.
49
50     Arguments
51     -----
52     f : callable
53         A function describing the discrete-time dynamics, such that
54         `s[k+1] = f(s[k], u[k])`.
55     s0 : numpy.ndarray
56         The initial state (1-D).

```

```

57     s_goal : numpy.ndarray
58         The goal state (1-D).
59     N : int
60         The time horizon of the LQR cost function.
61     Q : numpy.ndarray
62         The state cost matrix (2-D).
63     R : numpy.ndarray
64         The control cost matrix (2-D).
65     QN : numpy.ndarray
66         The terminal state cost matrix (2-D).
67     eps : float, optional
68         Termination threshold for iLQR.
69     max_iters : int, optional
70         Maximum number of iLQR iterations.
71
72     Returns
73     -----
74     s_bar : numpy.ndarray
75         A 2-D array where `s_bar[k]` is the nominal state at time step `k`,
76         for `k = 0, 1, ..., N-1`
77     u_bar : numpy.ndarray
78         A 2-D array where `u_bar[k]` is the nominal control at time step `k`,
79         for `k = 0, 1, ..., N-1`
80     Y : numpy.ndarray
81         A 3-D array where `Y[k]` is the matrix gain term of the iLQR control
82         law at time step `k`, for `k = 0, 1, ..., N-1`
83     y : numpy.ndarray
84         A 2-D array where `y[k]` is the offset term of the iLQR control law
85         at time step `k`, for `k = 0, 1, ..., N-1`
86     """
87     if max_iters <= 1:
88         raise ValueError("Argument `max_iters` must be at least 1.")
89     n = Q.shape[0] # state dimension
90     m = R.shape[0] # control dimension
91
92     # Initialize gains `Y` and offsets `y` for the policy
93     Y = np.zeros((N, m, n))
94     y = np.zeros((N, m))
95
96     # Initialize the nominal trajectory `(s_bar, u_bar)`, and the
97     # deviations `(ds, du)`
98     u_bar = np.zeros((N, m))
99     s_bar = np.zeros((N + 1, n))
100    s_bar[0] = s0
101    for k in range(N):
102        s_bar[k + 1] = f(s_bar[k], u_bar[k])
103    ds = np.zeros((N + 1, n))
104    du = np.zeros((N, m))
105
106    # iLQR loop
107    converged = False
108    for _ in range(max_iters):
109        # Linearize the dynamics at each step `k` of `(s_bar, u_bar)`
110        A, B = jax.vmap(linearize, in_axes=(None, 0, 0))(f, s_bar[:-1], u_bar)
111        A, B = np.array(A), np.array(B)
112
113        # PART (c) #####
114        # INSTRUCTIONS: Update `Y`, `y`, `ds`, `du`, `s_bar`, and `u_bar`.
115
116        P = QN

```

```

117     p = QN @ (s_bar[-1] - s_goal)
118     for k in range(N - 1, -1, -1):
119         inv = np.linalg.inv(R + B[k].T @ P @ B[k])
120         Y[k] = -inv @ B[k].T @ P @ A[k]
121         y[k] = -inv @ (R @ u_bar[k] + B[k].T @ p)
122         p = Q @ (s_bar[k] - s_goal) + A[k].T @ (p + P @ B[k] @ y[k])
123         P = Q + A[k].T @ P @ (A[k] + B[k] @ Y[k])
124
125     s_new = np.zeros_like(s_bar)
126     u_new = np.zeros_like(u_bar)
127     s_new[0] = s0
128     for k in range(N):
129         du[k] = Y[k] @ (s_new[k] - s_bar[k]) + y[k]
130         u_new[k] = u_bar[k] + du[k]
131         s_new[k + 1] = f(s_new[k], u_new[k])
132     s_bar = s_new
133     u_bar = u_new
134
135     #####
136
137     if np.max(np.abs(du)) < eps:
138         converged = True
139         break
140     if not converged:
141         raise RuntimeError("iLQR did not converge!")
142     return s_bar, u_bar, Y, y
143
144
145 def cartpole(s, u):
146     """Compute the cart-pole state derivative."""
147     mp = 2.0 # pendulum mass
148     mc = 10.0 # cart mass
149     L = 1.0 # pendulum length
150     g = 9.81 # gravitational acceleration
151
152     x, theta, dx, dtheta = s
153     sintheta, costheta = jnp.sin(theta), jnp.cos(theta)
154     h = mc + mp * (sintheta**2)
155     ds = jnp.array(
156         [
157             dx,
158             dtheta,
159             (mp * sintheta * (L * (dtheta**2) + g * costheta) + u[0]) / h,
160             -((mc + mp) * g * sintheta + mp * L * (dtheta**2) * sintheta * costheta + u[0] * costheta)
161             / (h * L),
162         ]
163     )
164     return ds
165
166
167 # Define constants
168 n = 4 # state dimension
169 m = 1 # control dimension
170 Q = np.diag(np.array([10.0, 10.0, 2.0, 2.0])) # state cost matrix
171 R = 1e-2 * np.eye(m) # control cost matrix
172 QN = 1e2 * np.eye(n) # terminal state cost matrix
173 s0 = np.array([0.0, 0.0, 0.0, 0.0]) # initial state
174 s_goal = np.array([0.0, np.pi, 0.0, 0.0]) # goal state
175 T = 10.0 # simulation time
176 dt = 0.1 # sampling time

```

```

177 animate = False # flag for animation
178 closed_loop = True # flag for closed-loop control
179
180 # Initialize continuous-time and discretized dynamics
181 f = jax.jit(cartpole)
182 fd = jax.jit(lambda s, u, dt=dt: s + dt * f(s, u))
183
184 # Compute the iLQR solution with the discretized dynamics
185 print("Computing iLQR solution ... ", end="", flush=True)
186 start = time.time()
187 t = np.arange(0.0, T, dt)
188 N = t.size - 1
189 s_bar, u_bar, Y, y = ilqr(fd, s0, s_goal, N, Q, R, QN)
190 print("done! ({:.2f} s)".format(time.time() - start), flush=True)
191
192 # Simulate on the true continuous-time system
193 print("Simulating ... ", end="", flush=True)
194 start = time.time()
195 s = np.zeros((N + 1, n))
196 u = np.zeros((N, m))
197 s[0] = s0
198 for k in range(N):
199     # PART (d) #####
200     # INSTRUCTIONS: Compute either the closed-loop or open-loop value of
201     # `u[k]`, depending on the Boolean flag `closed_loop`.
202     if closed_loop:
203         u[k] = u_bar[k] + Y[k] @ (s[k] - s_bar[k]) + y[k]
204     else: # do open-loop control
205         u[k] = u_bar[k]
206     #####
207     s[k + 1] = odeint(lambda s, t: f(s, u[k]), s[k], t[k : k + 2])[1]
208 print("done! ({:.2f} s)".format(time.time() - start), flush=True)
209
210 # Plot
211 fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
212 plt.subplots_adjust(wspace=0.45)
213 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
214 labels_u = (r"$u(t)$",)
215 for i in range(n):
216     axes[i].plot(t, s[:, i])
217     axes[i].set_xlabel(r"$t$")
218     axes[i].set_ylabel(labels_s[i])
219 for i in range(m):
220     axes[n + i].plot(t[:-1], u[:, i])
221     axes[n + i].set_xlabel(r"$t$")
222     axes[n + i].set_ylabel(labels_u[i])
223 if closed_loop:
224     plt.savefig("cartpole_swingup_cl.png", bbox_inches="tight")
225 else:
226     plt.savefig("cartpole_swingup_ol.png", bbox_inches="tight")
227 plt.show()
228
229 if animate:
230     fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
231     ani.save("cartpole_swingup.mp4", writer="ffmpeg")
232     plt.show()

```